ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba

Introducción al Aprendizaje Automático

Grado en Ingeniería Informática

Memoria de Prácticas

Autores: Jesús Fernández López
Rafael Carlos Schoettel Vilchez

Abril, 2026

Dedicado a
mi familia

Resumen

Esta memoria documenta el desarrollo y los resultados obtenidos en las sesiones experimentales de la asignatura Introducción al Aprendizaje Automático. El documento abarca el análisis empírico de algoritmos fundamentales, comenzando por la optimización mediante descenso por gradiente, donde se explora el impacto de la tasa de aprendizaje en la convergencia, la eficiencia computacional de los mini-batches y la implementación de criterios de parada. Posteriormente, se estudia el protocolo de Validación Cruzada (K-Fold) para la selección de modelos, así como las técnicas de regularización Ridge (L2) y Lasso (L1) para prevenir el sobreajuste y facilitar la selección automática de atributos.

Palabras clave: Aprendizaje Automático, Descenso por Gradiente, Validación Cruzada, Regularización, Ridge, Lasso.

Abstract

This report documents the development and results obtained during the experimental sessions of the course Introducción al Aprendizaje Automático. The document covers the empirical analysis of fundamental algorithms, starting with gradient descent optimization, where the impact of the learning rate on convergence, the computational efficiency of mini-batches, and the implementation of tolerance-based stopping criteria are explored. Subsequently, the K-Fold Cross-Validation protocol for model selection is studied, along with Ridge (L2) and Lasso (L1) regularization techniques to prevent overfitting and enable automatic feature selection.

Keywords: Machine Learning, Gradient Descent, Cross-Validation, Regularization, Ridge, Lasso.

Índice general

Índice de figuras

Índice de tablas

Capítulo 1

Cuaderno de Laboratorio — Práctica 1: La Física del Aprendizaje Automático

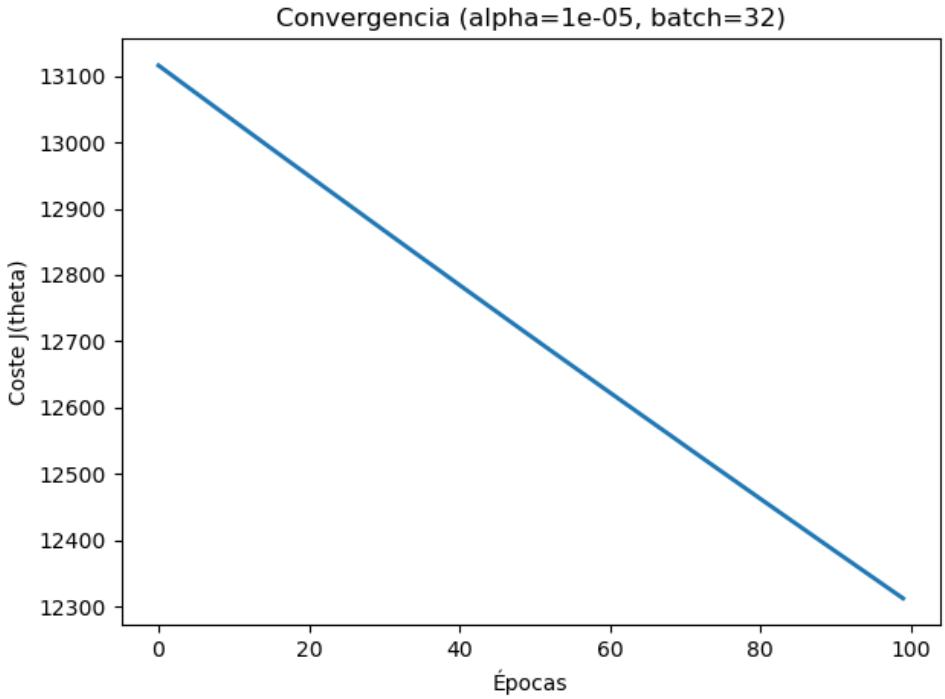
Objetivo

Comprender la dinámica de la optimización mediante gradiente descendente y el impacto de los hiperparámetros en la convergencia del algoritmo.

1.1. Tarea 1 — Importancia de la tasa de aprendizaje (α)

1.1.1. Escenario A (α_{slow}): convergencia lenta

Valor elegido: α = 1e-05

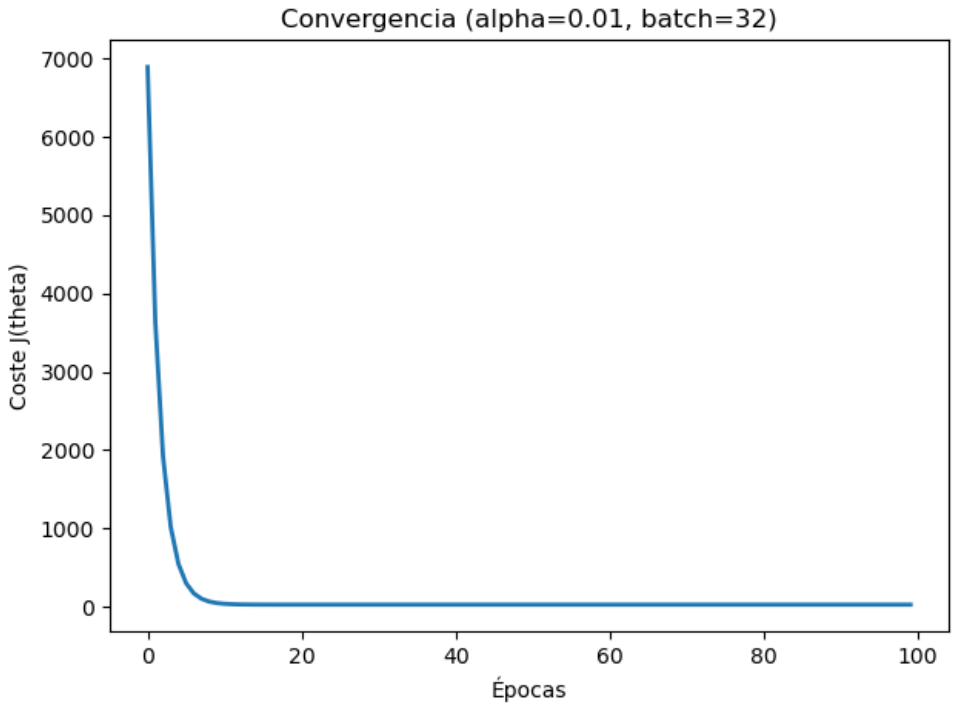


Análisis: ¿Por qué baja tan lentamente? ¿Cuántas iteraciones estimas para llegar cerca del mínimo?

Baja tan lentamente porque el tamaño de cada paso de actualización es minúsculo (el gradiente del error se multiplica por un α muy pequeño), por lo que los pesos apenas se modifican en cada iteración. Proyectando esta suave pendiente inicial, se estima que harían falta decenas de miles de iteraciones para acercarse de forma significativa al mínimo global.

1.1.2. Escenario B (α_{opt}): "curva de codo"

Valor elegido: $\alpha = 0.01$

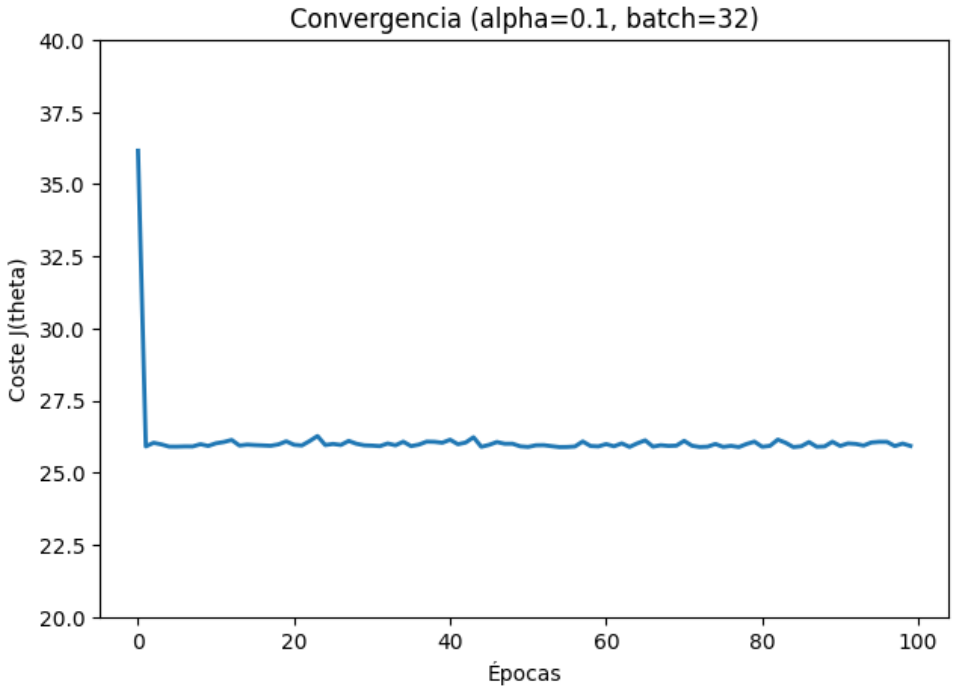


Análisis: Describe la "curva de codo" y qué te indica sobre rapidez y estabilidad.

La "curva de codo" muestra una caída inicial muy abrupta seguida de un aplanamiento drástico de la pendiente. Esto evidencia un comportamiento óptimo: el algoritmo avanza muy rápido cuando está lejos del mínimo y, al llegar al fondo del valle, los pasos del gradiente se ajustan milimétricamente sin rebotar, logrando así una gran rapidez en la convergencia y una estabilidad absoluta.

1.1.3. Escenario C (α_{osc}): oscilación amortiguada

Valor elegido: $\alpha = 0.1$

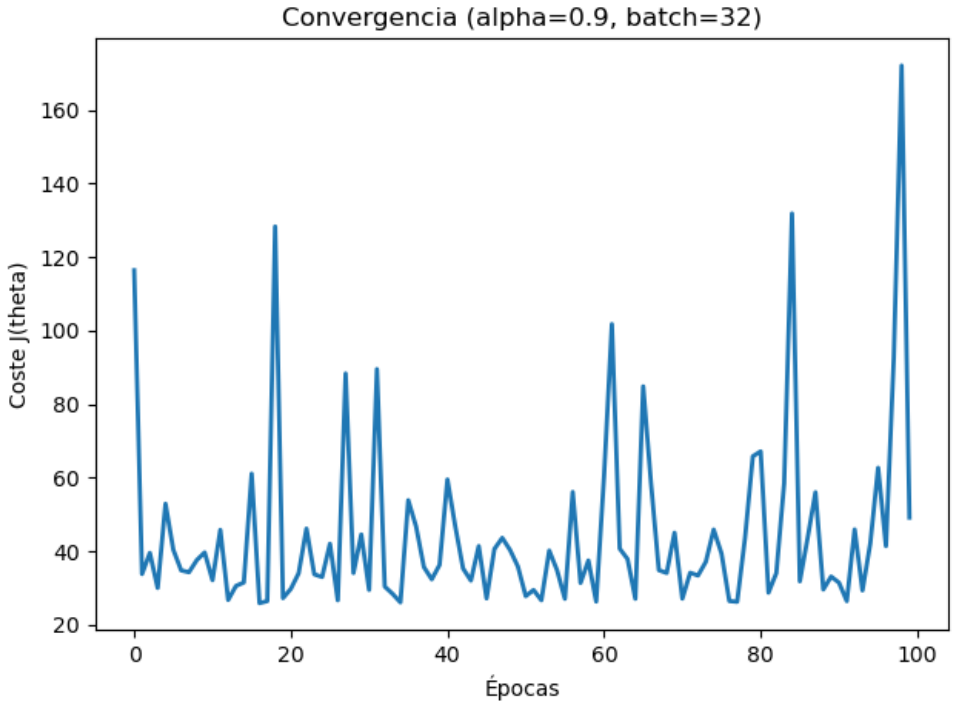


Análisis: ¿Qué ocurre con θ en el espacio de búsqueda? ¿Por qué oscila y puede estabilizarse?

En el espacio de búsqueda paramétrica, los pesos θ están sufriendo desplazamientos de magnitud excesiva y "saltan" repetidamente de una ladera a la opuesta del valle de coste, sobrepasando temporalmente el mínimo. Se observan oscilaciones por estos constantes rebotes; no obstante, el sistema logra estabilizarse porque la magnitud del verdadero gradiente disminuye gradualmente, atenuando la amplitud de los saltos hasta converger en la solución.

1.1.4. Escenario D (α_{fail}): divergencia

Valor elegido: $\alpha = 0.9$



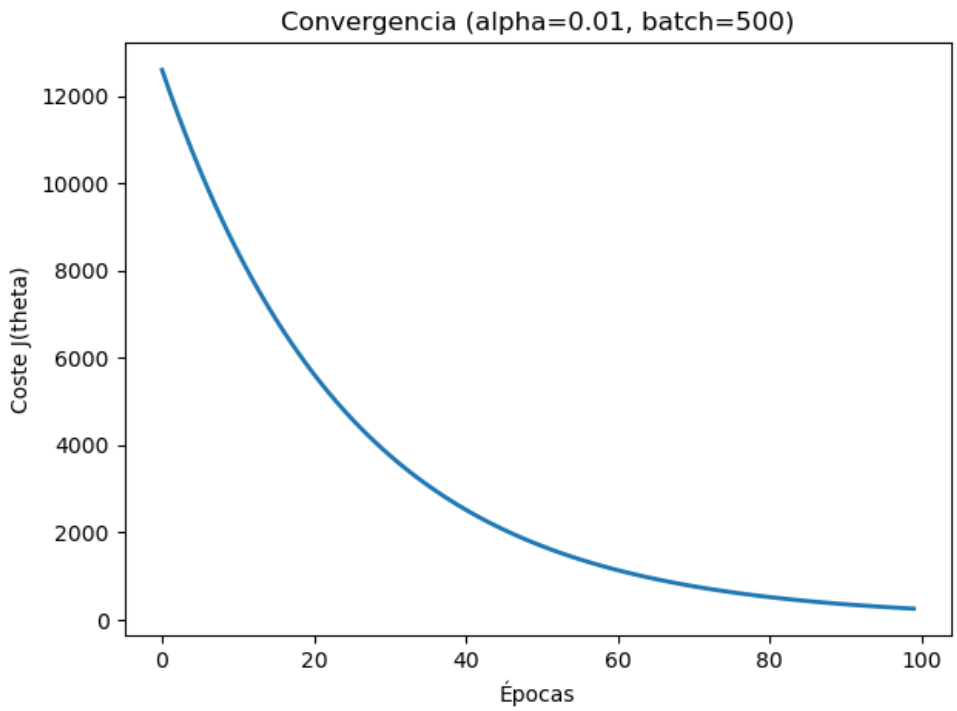
Análisis: ¿Por qué el error crece? Relaciónalo con "pasarse" del mínimo.

La función de coste diverge de forma monótona ascendente porque el coeficiente α es tan desproporcionado que el algoritmo "se pasa de largo" del mínimo local con una inercia destructiva. En lugar de descender gravitando hacia el valle, la gran magnitud de la corrección empuja a los pesos a rebotar cada vez a mayor altitud en las paredes opuestas, alejando indefinidamente la solución.

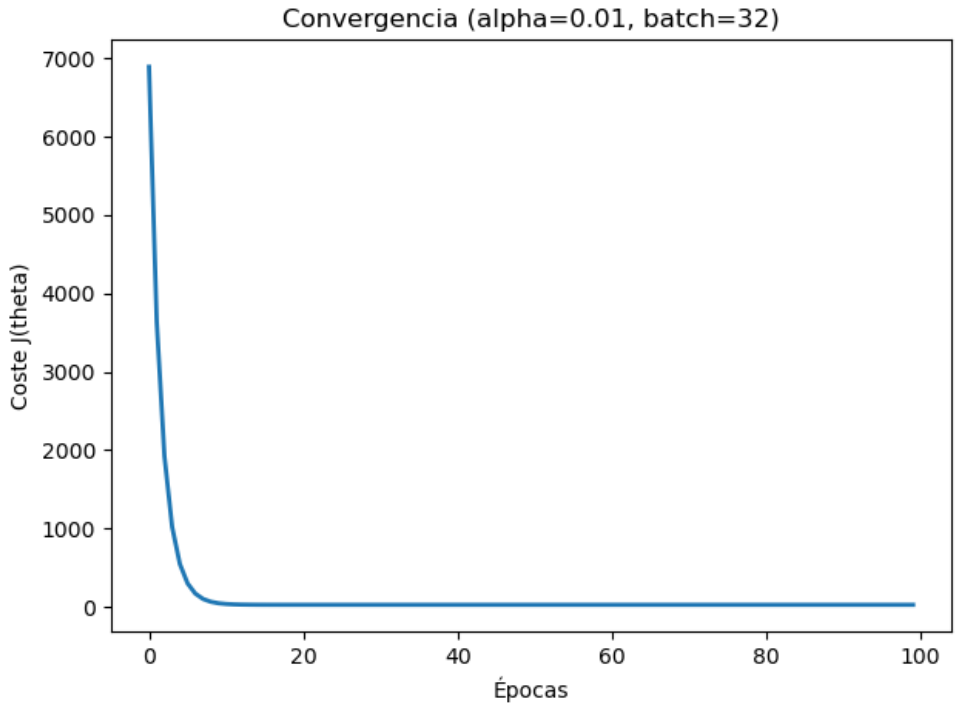
1.2. Tarea 2 — Compromiso del mini-batch

Usa tu α_{opt} y compara los siguientes esquemas de procesamiento:

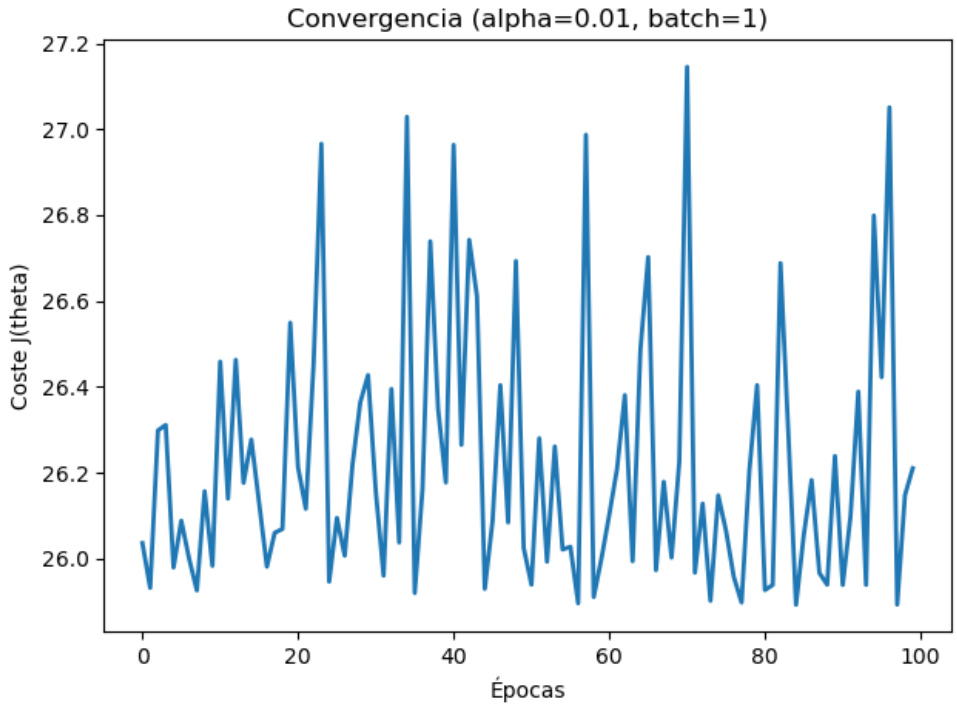
1.2.1. Batch completo



1.2.2. Mini-batch (32 o 16)



1.2.3. Estocástico puro (batch = 1)



Preguntas de reflexión:

1. ¿Cuál de las tres curvas es más "ruidosa" y por qué?

   La curva del gradiente estocástico puro (batch = 1) presenta el mayor nivel de ruido intrínseco. Fundamentar la dirección de máximo descenso en base a una única muestra observacional en cada iteración induce una elevadísima varianza estocástica. Esto provoca que el vector de descenso temporal apunte de forma muy errática, generando un fenotipo visual de dentado continuo (picos de subida y bajada). A pesar de esta inestabilidad local, la tendencia estocástica agregada garantiza un descenso a largo plazo.

2. A nivel de tiempo de ejecución (CPU), ¿cuál ha sido más eficiente? Justifica basándote en computación vectorial.

   A nivel hardware, tanto el Batch completo como el Mini-batch son radicalmente más eficientes que el descenso estocástico a base de iteraciones unitarias. Esto radica en el paradigma de la computación vectorial y las arquitecturas de instrucciones SIMD (Single Instruction, Multiple Data) del procesador. Agrupar los datos en bloques matriciales sustanciales permite a librerías numéricas de bajo nivel procesar transformaciones matemáticas complejas en estricto

paralelo y directamente en la ALU. Por el contrario, el modelo estocástico está condicionado a ejecutar un costoso control de flujo secuencial (bucles for ineficientes), provocando un cuello de botella abrumador en el tiempo de ejecución. El modelo **Mini-batch** resalta fundamentalmente por constituir un óptimo de Pareto: regulariza parcialmente el ruido de las muestras individuales garantizando un descenso suave, e invierte la carga computacional en veloces operaciones matriciales preservando simultáneamente una convergencia excepcionalmente rápida frente al Batch completo.

1.2.4. Tiempo de ejecución — Estocástico puro (batch = 1)

```
real    0m4.925s
user    0m4.886s
sys     0m0.039s
```

Figura 1.1: Tiempo de ejecución con batch=1.

1.2.5. Tiempo de ejecución — Mini-batch (batch = 32)

```
real    0m0.602s
user    0m0.568s
sys     0m0.033s
```

Figura 1.2: Tiempo de ejecución con batch=32.

1.2.6. Tiempo de ejecución — Batch completo (batch = 500)

```
real    0m0.495s
user    0m0.454s
sys     0m0.041s
```

Figura 1.3: Tiempo de ejecución con batch=500.

1.3. El reto — "Ajuste de precisión" (criterio de parada)

Implementa empíricamente el criterio de tolerancia algorítmica:

$$|J_t - J_{t-1}| < 10^{-5}$$

Para analizar las trayectorias asintóticas del sistema a largo plazo, se estableció un hiperparámetro deliberado que fijaba el límite en 2000 épocas en total. No obstante, al condicionar su diseño algorítmico al criterio de parada $|J_t - J_{t-1}| < 10^{-5}$ y equipar el modelo con su topología inicial más favorable ($\alpha = 0{,}01$, batch= 32), el algoritmo logra su punto de silla limitante ideal con una prontitud destacable y neutraliza la ejecución sistemáticamente en la **época 55**, obviando todo cómputo excedente e inútil que le sucediese al modelo a posterior.

```
real    0m0.435s
user    0m0.401s
sys     0m0.034s
```

Figura 1.4: Tiempo de convergencia interrumpida en la época 55 sujeto a $|J_t - J_{t-1}| < 10^{-5}$ para $\alpha = 0{,}01$ y batch= 32.

1.4. Conclusiones finales

El presente cuaderno de laboratorio corrobora categóricamente que la estabilidad e integridad de la convergencia global de un estimador iterativo requiere una rigurosa

sintonización analítica de sus hiperparámetros numéricos. Un factor paramétrico α marginal asegura la convergencia matemática, pero lo hace a expensas de un elevado lastre temporal infranqueable, mientras su hipertrofia causa la irremediable divergencia iterativa por retroproyección fuera del sumidero global. Por ello, el α óptimo impone un compromiso técnico de descenso inicial abrupto y mitigación suave sobre el vecindario absoluto. Respectivamente, se demuestra el segundo compromiso arquitectural: procesar muestras estocásticas individuales padece de extrema volatilidad métrica (ruido de las submuestras) y colapsa el motor computacional; el procesamiento de tipo Mini-batch emerge como el paradigma por excelencia superior, permitiendo actualizaciones continuas del vector de error al aprovechar el paralelismo subyacente de la microarquitectura del procesador de tipo vector (SIMD). Adicionalmente, el diseño del algoritmo ha reflejado categóricamente cómo implementar un control de flujo derivativo limitativo ($|J_t - J_{t-1}| < 10^{-5}$) extingue iteraciones marginales una vez garantizada la estabilización, culminando con un ahorro sustancial en el cálculo para el entrenamiento neuronal moderno.

Capítulo 2

Cuaderno de Laboratorio — Práctica 2: Selección de Modelos y Regularización

Objetivo

Dominar el protocolo de **Validación Cruzada (K-Fold)** y comprender cómo la regularización **Ridge (L2)** y **Lasso (L1)** previenen el sobreajuste y ayudan a la **selección de atributos**.

2.1. Tarea 1 — Estabilidad de la Validación Cruzada (K-Fold)

Configuración común

Modelo usado: **Regresión Lineal (Ridge con $\lambda \approx 0$)** (regresión lineal sin regularización)
Métrica: **RMSE** (RMSE)
Semillas: **`random_state` 1, 2, 3, 4 y 5, de forma consecutiva para la tarea 1, y el valor 42 para el resto.**

Hipótesis inicial (antes de ejecutar): ¿Crees que el RMSE será estable al cambiar K? No. Con valores muy bajos de K (ej. K=2), el error dependerá en gran medida de qué datos caigan aleatoriamente en entrenamiento y cuáles en test, provocando alta varianza.

2.1.1. Experimento A — K = 2 (Validación simple): variabilidad entre ejecuciones

Ejecuta el modelo **varias veces** con K=2 (por ejemplo 5–10) y registra el RMSE.

```
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.33 | Time=0.0046s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.43 | Time=0.0050s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.24 | Time=0.0055s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.17 | Time=0.0046s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.54 | Time=0.0057s
```

Registro de resultados (rellena):

Ejecución	RMSE
1	5.33
2	5.43
3	5.24
4	5.17
5	5.54

Análisis:

- ¿Por qué el error varía tanto entre ejecuciones? Porque el conjunto de entrenamiento es pequeño (50 % de los datos). El modelo es extremadamente sensible a la partición aleatoria específica de cada ejecución.
- ¿Es fiable esta métrica (con K=2) para tomar una decisión técnica? No, su alta varianza impide saber si una mejora aparente se debe a la configuración del modelo o simplemente a la aleatoriedad de la división de datos.

2.1.2. Experimento B — K = 10 (estándar): RMSE más estable

Ejecuta el modelo con K=10 y anota el RMSE.

```
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.21 | Time=0.0190s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.19 | Time=0.0135s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.16 | Time=0.0154s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.17 | Time=0.0179s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.23 | Time=0.0161s
```

RMSE observado (K=10): **5.19 (Media aprox.)**

Análisis: Describe si el resultado parece más estable que en K=2 y por qué. Sí, es notablemente más estable (fluctúa entre 5.16 y 5.23). Al usar K=10, el modelo entrena con el 90 % de los datos en cada iteración y promedia 10 errores diferentes, mitigando drásticamente el impacto de particiones anómalas.

2.1.3. Experimento C — K = 100 (Leave-One-Out): coste vs. precisión

Ejecuta el modelo con K=100 (LOOCV) y observa el RMSE y el tiempo.

```
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.18 | Time=0.1252s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.18 | Time=0.1395s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.17 | Time=0.1390s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.18 | Time=0.1290s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.19 | Time=0.1231s
```

RMSE observado (K=100): **5.18** Tiempo aproximado: **0.131s**

Análisis: Compara el coste computacional frente a K=10. El tiempo de ejecución se multiplica casi por 8 (de $\sim$ 0,016s a $\sim$ 0,131s). ¿Merece la pena el tiempo extra para mejorar la precisión obtenida? No merece la pena. La mejora en la precisión del RMSE es marginal (apenas 0.01 unidades), lo cual no justifica el enorme aumento del coste computacional.

2.1.4. Resumen comparativo (K = 2 vs 10 vs 100)

K	RMSE	Comentario de estabilidad/coste
2	5.17 - 5.54	Alta inestabilidad (varianza). Coste muy bajo.
10	5.16 - 5.23	Alta estabilidad. Coste equilibrado y eficiente.
100	5.17 - 5.19	Máxima estabilidad. Coste computacional prohibitivo.

Conclusión Tarea 1: ¿Cuál elegirías para este problema y por qué? Elegiría K=10. Demuestra ser el estándar óptimo porque ofrece un error robusto y fiable para comparar modelos, sin incurrir en los altos tiempos de ejecución que exige K=100. Proporciona el mejor compromiso técnico entre precisión estadística y eficiencia temporal.

2.2. Tarea 2 — Regularización Ridge vs. Lasso (K = 10)

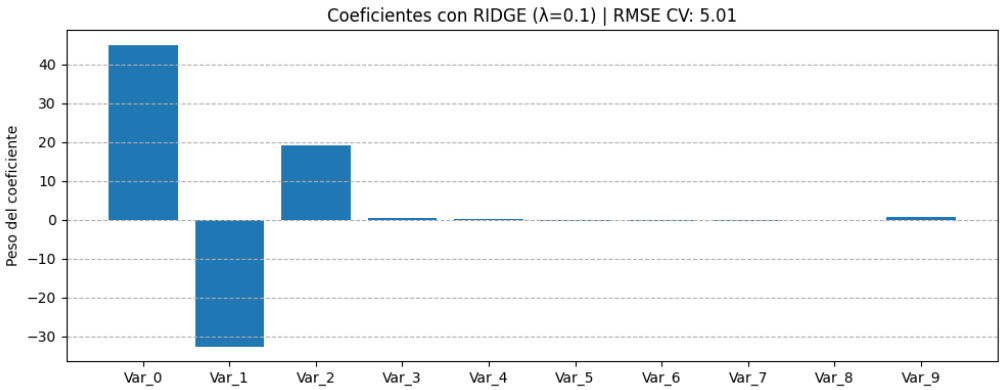
Configuración

Fijamos K = 10 y comparamos:

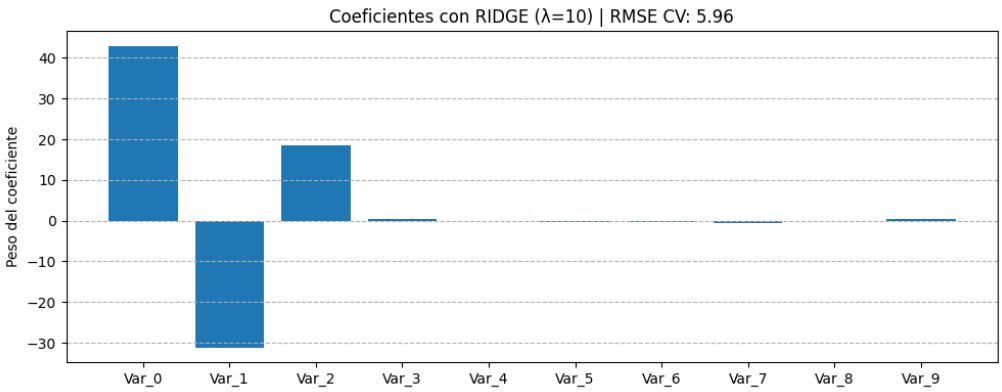
- **Ridge Regression** (penalización L2)
- **Lasso Regression** (penalización L1)

2.2.1. Ridge (L2) — $\lambda = 0{,}1,\ 10,\ 100$

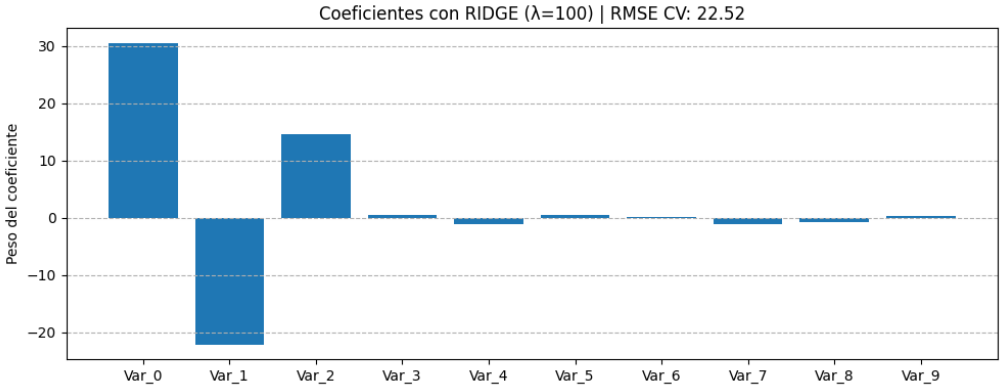
Resultados: captura el gráfico de barras de coeficientes para cada λ.



RMSE Ridge ($\lambda = 0{,}1$): **5.01**



RMSE Ridge ($\lambda = 10$): **5.96**

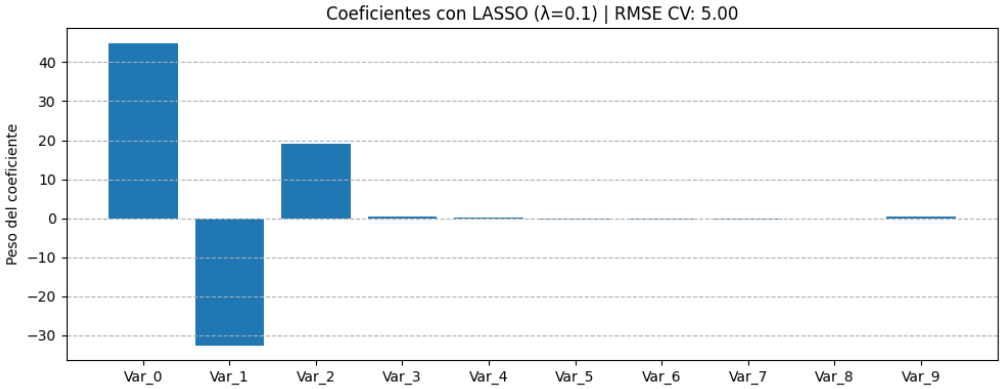


RMSE Ridge ($\lambda = 100$): **22.52**

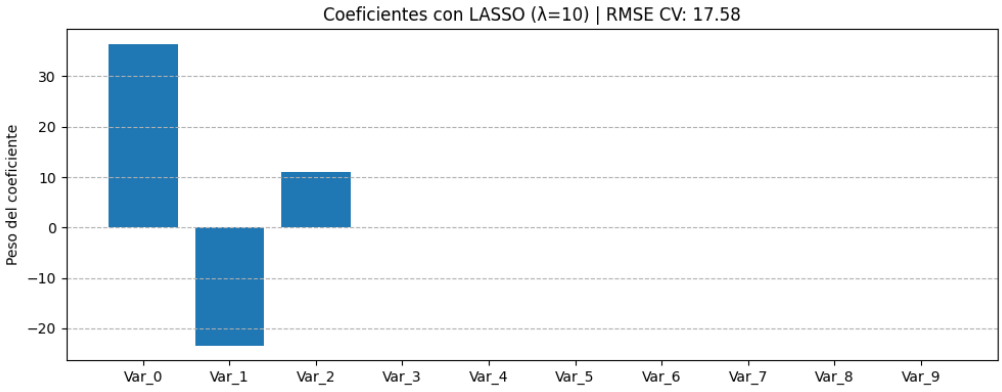
Análisis: ¿Llegan a valer **cero** los coeficientes de las variables ruidosas?. No llegan a valer cero absoluto. Visualmente en las gráficas parece que algunas como (`Var_8` desaparecen, pero técnicamente Ridge (L2) simplemente reduce o .^en^coge"drásticamente la magnitud de los coeficientes penalizados, distribuyendo el peso entre todos sin llegar a anularlos por completo.

2.2.2. Lasso (L1) — $\lambda = 0{,}1,\ 10,\ 100$

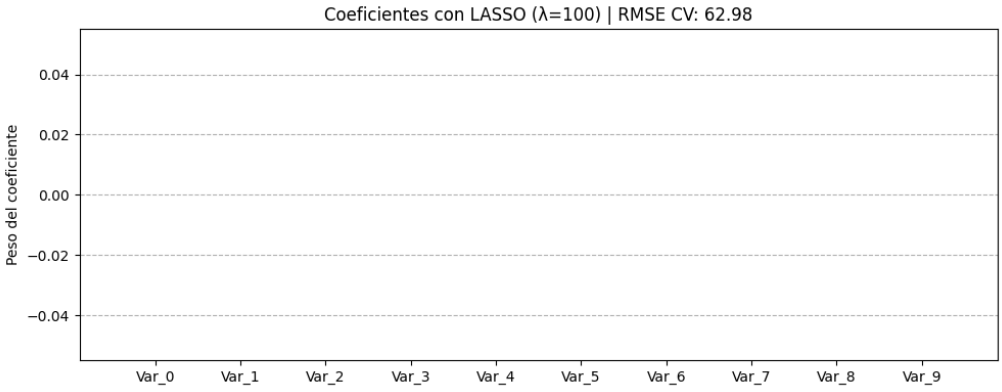
Resultados: captura el gráfico de barras de coeficientes para cada λ.



RMSE Lasso ($\lambda = 0{,}1$): **5.00**



RMSE Lasso ($\lambda = 10$): **17.58**



RMSE Lasso ($\lambda = 100$): **62.98**

Análisis clave:

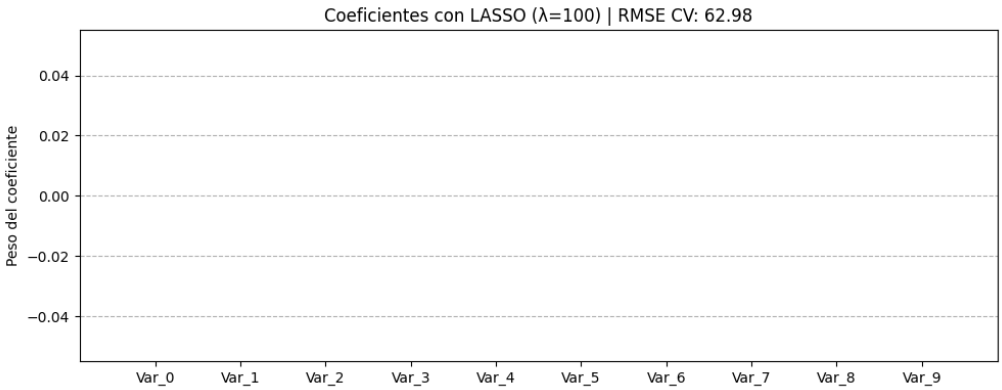
1. Identifica para qué valor de λ el modelo **Lasso** consigue **eliminar** (poner a cero) las 7 variables ruidosas, quedándose solo con las 3 importantes:

$$\lambda^* = \mathbf{1.0}(\text{y en general entre 0.5 y 2.0})$$

2. Variables con coeficiente distinto de cero (modelo Lasso con λ^*):
Var_0, Var_1, Var_2

2.2.3. El dilema de la complejidad — λ demasiado alto

Prueba un valor extremo, por ejemplo $\lambda = 100$, y registra qué ocurre.



RMSE ($\lambda = 100$): **62.98**

Análisis: Explica qué ocurre con el RMSE si aplicas un λ demasiado alto. El RMSE se dispara exageradamente (hasta 62.98). ¿Estamos ante un caso de **sesgo alto** o **varianza alta**? Justifica. Estamos ante un caso evidente de **sesgo alto (underfitting)**. La penalización es tan severa que el modelo se vuelve excesivamente rígido, empujando todos los coeficientes hacia cero e impidiendo que pueda capturar la señal o patrón real de los datos.

2.3. Reto — El "Cazador de Variables"

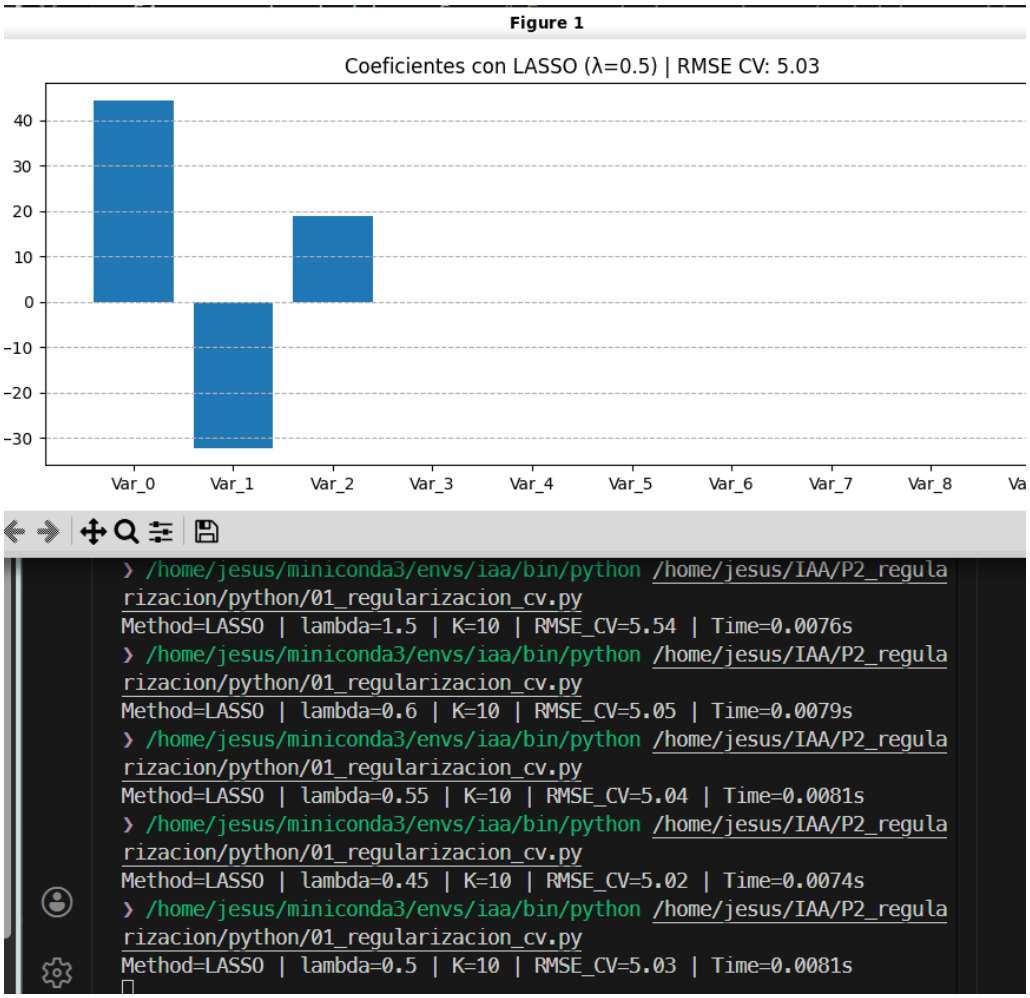
Contexto: Imagina que cada variable incluida en tu modelo tiene un **coste de mantenimiento**. Tu objetivo es encontrar el modelo **más sencillo posible** (menos variables con coeficientes $\neq 0$) que mantenga un RMSE inferior a un umbral:

$$\text{RMSE} < \mathbf{5.50} \quad \text{(umbral asumido)}$$

Mejor solución encontrada (rellena):

$$\lambda = \mathbf{0.5} \qquad \#\text{Variables activas} = \mathbf{3} \qquad \text{RMSE} = \mathbf{5.03}$$

Variables seleccionadas (coeficiente $\neq 0$): **Var_0, Var_1 y Var_2**



Justificación técnica (6–10 líneas): ¿Por qué este modelo es el mejor compromiso entre simplicidad (coste) y rendimiento (RMSE)? Tras realizar un ajuste paramétrico fino, $\lambda = 0{,}5$ demuestra ser el punto de equilibrio óptimo. Logra la máxima simplicidad técnica al descartar completamente los 7 atributos que solo aportaban ruido, minimizando así los futuros costes de mantenimiento e ingesta de datos del modelo. Simultáneamente, conserva un rendimiento excelente con un RMSE de 5.03, el cual es prácticamente idéntico al de modelos menos restrictivos (como $\lambda = 0{,}1$ con RMSE 5.00). Nos quedamos con la señal real y descartamos el ruido sin penalizar el error.

2.4. Conclusiones finales

Resume en 8–12 líneas lo aprendido sobre:

- Estabilidad de K-Fold al variar K (K=2 vs K=10 vs LOOCV).
- Diferencias prácticas entre Ridge y Lasso (coeficientes, selección de variables).
- Relación entre λ grande y el sesgo/varianza.

La validación cruzada con $K = 10$ es el estándar porque equilibra un RMSE estadísticamente fiable frente a la inestabilidad de $K = 2$, evitando los desproporcionados tiempos de ejecución de LOOCV ($K = 100$). En cuanto a la regularización, Ridge (L2) reduce la magnitud de los coeficientes de las variables menos relevantes pero los mantiene activos, mientras que Lasso (L1) es capaz de forzarlos a cero absoluto. Esta propiedad hace que Lasso actúe como un potente selector automático de características, ideal para limpiar datasets con ruido. Por último, un hiperparámetro λ demasiado alto penaliza en exceso al modelo, volviéndolo incapaz de aprender (underfitting) y empujándolo a una zona de sesgo alto donde el error se dispara.

Capítulo 3

Cuaderno de Laboratorio — Práctica 3: Metodología de Evaluación y Rigor Científico

Objetivo

Aprender a realizar evaluaciones honestas en condiciones difíciles, especialmente con datos desbalanceados, y detectar errores metodológicos graves como la fuga de datos (*data leakage*).

Material de partida

Se parte de:

- Un dataset sintético de **1000 pacientes** con **5 variables predictoras**.
- Una clase positiva muy minoritaria (**2 %** de los casos), es decir, $K = 20$ de $N = 1000$.

3.1. Tarea 1 — La lotería de la partición aleatoria

Configuración común

Modelo usado: `LogisticRegression` (con `class_weight='balanced'`)
Métricas usadas: `Nº positivos en test`, `% clase 1`
Semilla(s): `0, 1, 2, 3, 4` (5 ejecuciones aleatorias)

Hipótesis inicial: Es probable que el número de positivos en test sea muy inestable. Al haber solo un 2 % de casos positivos (20 de 1000), un split aleatorio del 20 % (200 ejemplos) podría coger muy pocos positivos por puro azar, o incluso ninguno. Esto dejaría el test inservible.

3.1.1. Análisis estadístico de la partición

Para comprobar el riesgo real de hacer una mala partición, he calculado la probabilidad usando la distribución hipergeométrica. Teniendo $N = 1000$ pacientes, $K = 20$ positivos en total, y sacando una muestra para test de $n = 200$:

Lo normal (la media esperada) es que caigan 4 positivos en test:

$$\mathbb{E}[X] = \frac{200 \cdot 20}{1000} = 4$$

Pero, ¿qué pasa si tenemos mala suerte? Calculando la probabilidad de que no caiga **ningún** positivo ($X = 0$):

$$P(X = 0) = \frac{\binom{20}{0}\binom{980}{200}}{\binom{1000}{200}} \approx 0{,}0110 \quad (1{,}10\,\%)$$

Si sumamos la probabilidad de sacar 0 o 1 positivo, el riesgo sube a casi un 7 %:

$$P(X \leq 1) \approx 0{,}0110 + 0{,}0563 = 0{,}0673 \quad (6{,}73\,\%)$$

¿Por qué es un problema? Si en el test no hay positivos ($X = 0$), las métricas se rompen:

- **Recall**: No se puede calcular (0/0) porque no hay verdaderos positivos ni falsos negativos reales.
- **F1-score**: Al depender del Recall, también da error.
- **AUC-ROC**: La librería devuelve `NaN` porque necesita ejemplos de ambas clases para pintar la curva.

Conclusión: un mal split aleatorio puede hacer que la evaluación no sirva para nada, sin importar qué modelo usemos.

3.1.2. Registro de resultados (5 particiones aleatorias)

Ejecución	Positivos en test	% clase 1 en test
1	4	2,0 %
2	3	1,5 %
3	5	2,5 %
4	4	2,0 %
5	3	1,5 %

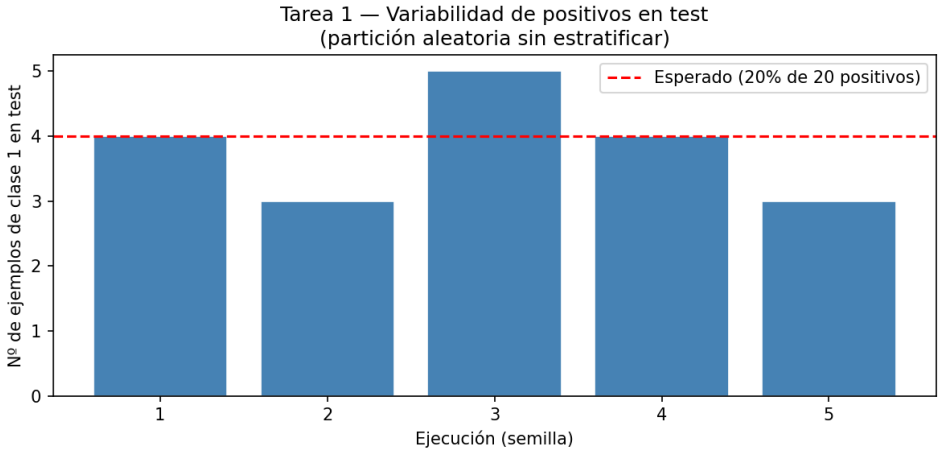


Figura 3.1: Variabilidad del número de ejemplos positivos en el conjunto de test según la semilla. La línea roja indica el valor esperado teórico (4).

Análisis de los resultados:

- *¿Ha cambiado mucho el número de positivos en test entre ejecuciones?* Sí, el número oscila entre 3 y 5 en mis ejecuciones. Aunque numéricamente parezca poco, en un test tan pequeño supone una variación del ±25 %, lo que altera bastante las métricas.

- *¿Sería fiable la evaluación si el azar dejara solo 0 o 1 positivos en test?* Para nada. Como he demostrado antes, hay un ≈ 6,73 % de probabilidad de que ocurra. Con 0 positivos las métricas dan error matemático, y con 1 solo positivo el resultado dependería al 100 % de acertar o fallar en un único paciente, perdiendo todo el rigor estadístico.

3.2. Tarea 2 — Partición estratificada

3.2.1. Registro por folds (`StratifiedKFold`, $k = 10$)

Fold	Positivos en test	Tamaño del fold	Proporción clase 1
1	2	100	2,0 %
2	2	100	2,0 %
3	2	100	2,0 %
4	2	100	2,0 %
5	2	100	2,0 %
6	2	100	2,0 %
7	2	100	2,0 %
8	2	100	2,0 %
9	2	100	2,0 %
10	2	100	2,0 %

Desviación típica de la proporción de clase 1 entre folds: **0,0000 %** (prácticamente nula, lo que confirma que la estratificación funciona correctamente).

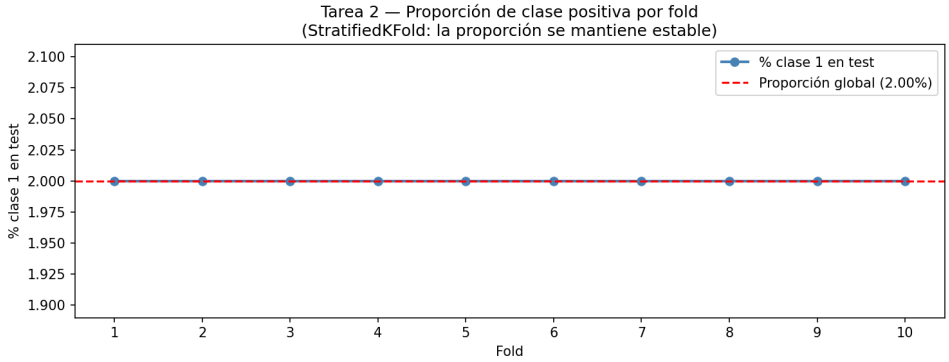


Figura 3.2: Proporción de la clase positiva por fold con `StratifiedKFold`. La línea roja indica la proporción global del dataset (2 %).

Análisis: `StratifiedKFold` garantiza que cada fold mantenga la misma proporción de clases que el dataset original (2 %). En datasets muy desbalanceados, esto evita que algunos folds se queden sin muestras de la clase minoritaria, asegurando que el modelo sea evaluado siempre sobre una distribución representativa y reduciendo la varianza debida al azar.

3.3. Tarea 3 — Comparativa de la varianza

3.3.1. Resumen de resultados (10 iteraciones, 10 folds)

La tabla **??** recoge la media y la desviación típica de las tres métricas calculadas mediante `cross_val_score` durante 10 iteraciones independientes de validación cruzada con 10 *folds*. He añadido la métrica AUC-ROC para reforzar las conclusiones sobre la necesidad de estratificación.

Tabla 3.1: Comparativa de estabilidad de métricas: KFold estándar vs. StratifiedKFold (10 iteraciones × 10 folds).

Método	**Accuracy**		**F1-macro**		**AUC-ROC**	
	Media	*std*	Media	*std*	Media	*std*
`KFold`	0,5771	0,0090	0,3829	0,0051	**N/D** (*8/10 iters.*†)	
`StratifiedKFold`	0,5783	**0,0085**	0,3857	**0,0050**	**0,5283**	**0,0253**

† **N/D = No Definida.** En 8 de las 10 iteraciones de `KFold`, algún fold del conjunto de test careció de ejemplos positivos. En esas condiciones, `roc_auc_score` devuelve `NaN` (indefinido), puesto que la curva ROC no puede construirse sin ambas clases representadas. Solo en las 2 iteraciones en que por azar todos los folds tuvieron al menos un positivo se obtuvo un valor numérico (AUC ≈ 0,49). Este resultado es la prueba empírica más directa de la necesidad de estratificación: sin ella, la métrica de evaluación más robusta para clases desbalanceadas queda matemáticamente indefinida en la mayoría de los experimentos.

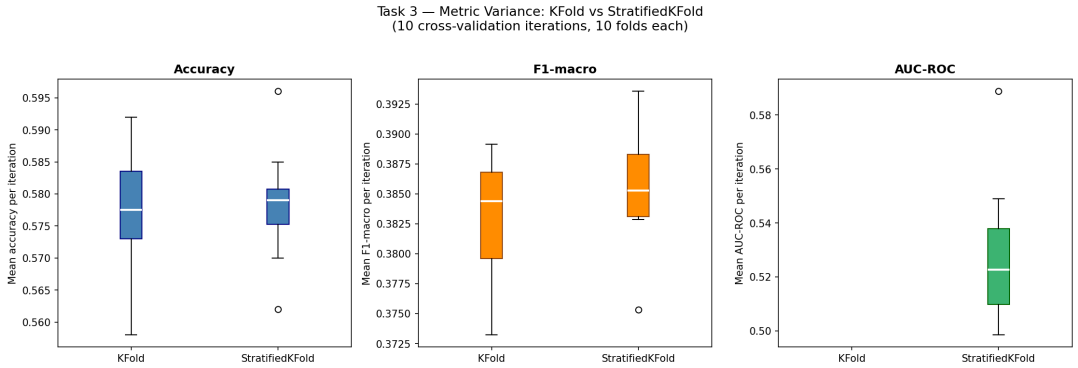


Figura 3.3: Distribución de Accuracy, F1-macro y AUC-ROC a lo largo de 10 iteraciones de validación cruzada (10 *folds*). Cada métrica se muestra como un diagrama de cajas (*boxplot*). `StratifiedKFold` produce cajas más estrechas en las tres métricas, evidenciando menor varianza y mayor estabilidad en la evaluación.

Interpretación:

- *¿Qué método produce métricas más estables?* El método `StratifiedKFold` produce métricas más estables, como se observa en la menor desviación típica (*std*) en Accuracy y F1-macro; pero la diferencia más llamativa se produce en **AUC-ROC**.

- *¿Dónde se nota más la mejora?* En AUC-ROC, sin duda. Con `KFold estándar`, la métrica es **indefinida (`NaN`)** en 8 de las 10 iteraciones, porque algunos folds quedan vacíos de ejemplos positivos. Solo por azar, en 2 iteraciones se obtiene un valor numérico ($\approx$ 0,49), que además es desfavorable (cercano a una clasificación aleatoria). Con `StratifiedKFold`, la AUC-ROC es siempre computable: media = 0,5283, desv. típica = 0,0253, en las 10 iteraciones.

- **La AUC-ROC reafirma empíricamente la necesidad de estratificar:** el hecho de que `KFold` haga la métrica indefinida en el 80 % de los experimentos no es un artefacto numérico, sino la *manifestación directa* del problema demostrado matemáticamente en la Tarea 1 mediante la distribución hipergeométrica: cuando los folds no están estratificados, es probable que algunos queden sin representantes de la clase positiva, haciendo imposible cualquier evaluación orientada a esa clase. La AUC-ROC, al ser la métrica más exigente en ese sentido, lo pone de manifiesto de forma inequívoca.

3.4. Tarea 4 — Detección de *data leakage*

3.4.1. Comparativa: con y sin variable trampa

Escenario	Accuracy	F1-macro
Sin variable trampa (*evaluación real*)	0,5850	0,3887
Con variable trampa (`ID_Hospital_Filtro`)	**1,0000**	**1,0000**

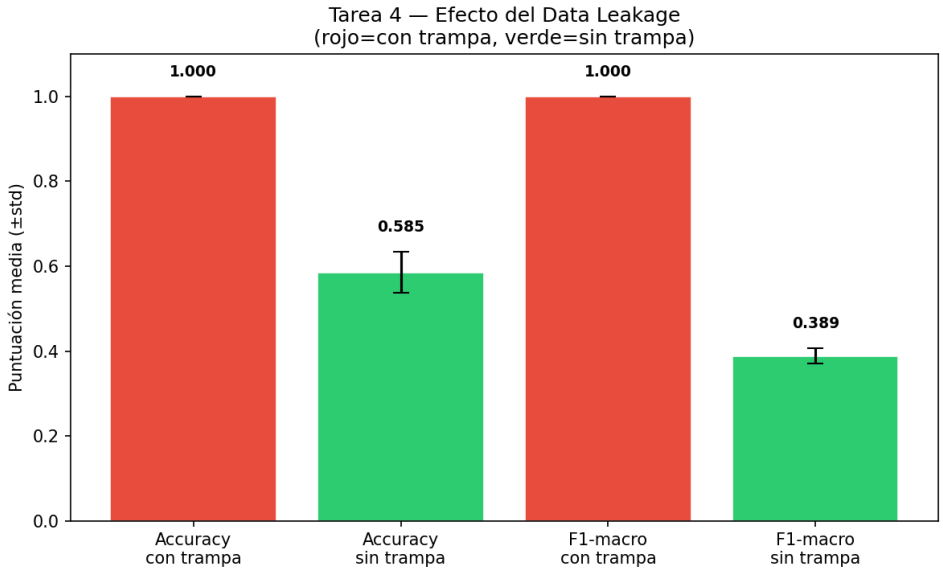


Figura 3.4: Efecto del *data leakage*. Las barras rojas corresponden al escenario con la variable trampa; las verdes, al escenario real sin ella.

Preguntas:

- *¿Qué variable está filtrando información del objetivo?* La variable `ID_Hospital_Filtro`.
- *¿Por qué esa variable invalida la evaluación?* Ha sido construida con una correlación perfecta (o casi perfecta) con la clase objetivo. El modelo aprende a usar este "filtro" en lugar de las variables clínicas reales, obteniendo un rendimiento del 100 % que es imposible en la práctica real.
- *¿Diferencia entre un modelo "bueno" y uno contaminado por leakage?* A diferencia de un modelo legítimo, uno con leakage simplemente está haciendo trampa usando información que no estará disponible en producción, por lo que su 100 % de acierto es un espejismo.

3.5. Reto — Auditoría metodológica

1. **Partición aleatoria:** Como hemos comprobado, corremos el riesgo de dejar el test sin apenas casos positivos, lo que invalida cualquier métrica que intentemos calcular.

2. **Estratificación:** Es la solución directa al problema anterior, ya que nos garantiza una representación mínima de la clase minoritaria en cada fold para poder evaluar de forma justa.

3. **Fuga de datos (Leakage):** Invalida por completo el experimento. El modelo no aprende, solo usa la variable trampa como atajo, por lo que fallará estrepitosamente en el mundo real.

4. **Buenas prácticas:** Usar siempre estratificación, auditar variables en busca de correlaciones sospechosas, emplear métricas robustas (F1, AUC-ROC) y documentar la reproducibilidad mediante semillas y validación cruzada repetida.

3.6. Conclusión final

Esta práctica me ha dejado claro que evaluar un modelo no es solo ejecutar unas líneas de código y mirar la *Accuracy*: con datos desbalanceados, hay que tomar decisiones de diseño concretas o los números que obtenemos nos van a engañar.

En cuanto a la **partición**, calculé con la distribución hipergeométrica que hay un $P(X = 0) \approx 1{,}10\,\%$ de probabilidad de que el test quede sin ningún positivo, y casi un 7 % de terminar con uno solo. En esas condiciones, F1-score, Recall y AUC-ROC son directamente indefinidas. Eso ya es razón suficiente para usar siempre estratificación: no es opcional si queremos que los resultados tengan sentido.

Los resultados obtenidos con 10 iteraciones de validación cruzada lo confirman: con `KFold` estándar, la AUC-ROC fue `NaN` en 8 de las 10 iteraciones porque algunos folds se quedaron sin positivos; con `StratifiedKFold` siempre fue computable y más estable (*std* menor en las tres métricas).

En cuanto al **data leakage**, el caso de `ID_Hospital_Filtro` es bastante claro: una variable artificialmente correlacionada con el target hace que el modelo alcance un 100 % de Accuracy, lo que en la práctica significaría entregar un sistema que falla en cuanto se usa con datos reales. La regla práctica que saco de aquí es auditarlas siempre antes de entrenar.

Capítulo 4

Cuaderno de Laboratorio — Práctica 4: Clasificación y el Valor del Error (Matriz de Costes)

Objetivo

Entrenar un clasificador probabilístico para un problema de detección de fraude bancario y estudiar cómo el umbral de decisión afecta a los errores cometidos y al coste total. El objetivo central no es únicamente medir el rendimiento predictivo, sino comprender el impacto económico real de las decisiones del modelo a partir de una **matriz de costes asimétrica**.

Descripción del problema

Se trabaja con un problema de clasificación binaria sobre el dataset `fraude_transacciones.cs`

- **Clase 0**: transacción normal.
- **Clase 1**: transacción fraudulenta.

Los errores tienen costes muy distintos:

- **Falso Negativo (FN)**: no detectar un fraude → 200 €
- **Falso Positivo (FP)**: bloquear una transacción legítima → 10 €

La decisión final no debe evaluarse solo por el número de aciertos, sino por el **coste económico** asociado a los errores.

4.1. Tarea 1 — Análisis con el umbral por defecto ($\tau = 0{,}5$)

4.1.1. Modelo utilizado

Se ha entrenado una **Regresión Logística** (`LogisticRegression`, `max_iter=1000`) sobre el 70 % del dataset (partición estratificada, `random_state=42`) y evaluado sobre el 30 % restante.

4.1.2. Matriz de confusión

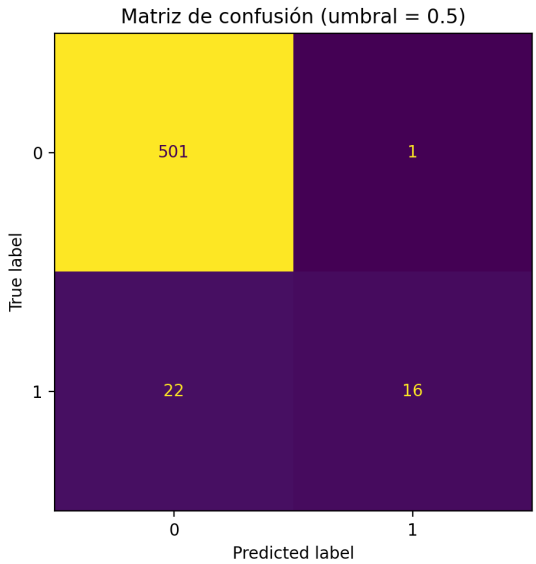


Figura 4.1: Matriz de confusión para el umbral estándar $\tau = 0{,}5$.

A partir de esta matriz se obtienen:

- Verdaderos Positivos (TP): **16**
- Verdaderos Negativos (TN): **501**
- Falsos Positivos (FP): **1**
- Falsos Negativos (FN): **22**

4.1.3. Cálculo del coste total

$$\text{Coste Total} = (FP \cdot 10) + (FN \cdot 200) = (1 \cdot 10) + (22 \cdot 200) = \mathbf{4410}\,€$$

4.1.4. Interpretación

El umbral por defecto de 0.5 **no es razonable** en este contexto. Aunque el modelo comete muy pocos falsos positivos (solo 1), deja pasar 22 transacciones fraudulentas. Dado que el coste de un falso negativo (200 €) es veinte veces el de un falso positivo (10 €), el impacto económico resulta muy elevado. En un sistema de detección de fraude es fundamental minimizar los errores más caros; usar este umbral sin revisarlo sería un error de diseño grave.

4.2. Tarea 2 — Estudio de probabilidades y umbrales

4.2.1. Análisis de distintos umbrales

Se han extraído las probabilidades de la clase fraude para cada observación y se han evaluado umbrales $\tau \in [0, 1]$ en pasos de 0.05. La Tabla **??** resume los resultados para los primeros tramos y el extremo superior.

Tabla 4.1: Resumen de FP, FN y Coste Total para distintos umbrales de decisión.

Umbral	FP	FN	Coste Total (€)
0.00	502	0	5020
0.05	112	9	**2920**
0.10	42	14	3220
0.15	22	15	3220
0.20	13	15	3130
0.25	9	16	3290
0.30	4	17	3440
0.40	1	20	4010
0.50	1	22	4410
1.00	0	38	7600

4.2.2. Observaciones

Existe un claro compromiso (*trade-off*) entre los dos tipos de error al variar τ:

- **Cuando el umbral baja** (hacia 0): se exige menos confianza para clasificar como fraude. Caen los Falsos Negativos (detectamos más fraudes), pero aumentan drásticamente los Falsos Positivos (bloqueamos más transacciones legítimas).
- **Cuando el umbral sube** (hacia 1): las alertas son mucho más selectivas. Los Falsos Positivos desaparecen, pero los Falsos Negativos se disparan (dejamos escapar la mayoría de los fraudes).

El umbral de decisión debe ajustarse según la penalización económica de cada error, en lugar de asumir un valor estático por defecto.

4.3. Tarea 3 — Optimización del coste

4.3.1. Gráfica de coste frente a umbral

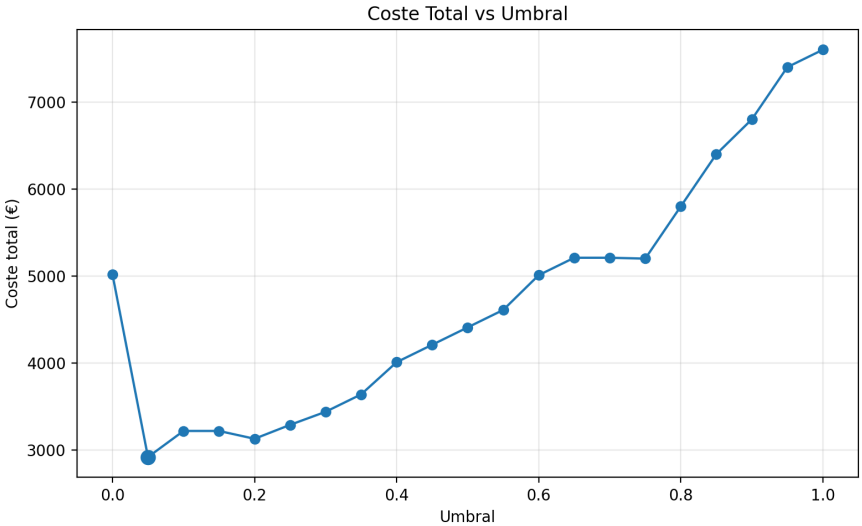


Figura 4.2: Coste total (€) en función del umbral de decisión. El punto óptimo se alcanza en $\tau^* = 0{,}05$.

4.3.2. Umbral óptimo

$$\tau^* = 0{,}05 \qquad \text{Coste mínimo} = 2920\,€$$

4.3.3. Interpretación

El umbral óptimo ($\tau^* = 0{,}05$) es muy inferior a 0.5. Esto tiene todo el sentido dada la fuerte asimetría de la matriz de costes: el coste de un FN (200 €) es 20 veces el de un FP (10 €). Para minimizar el coste total conviene ser muy sensible a los fraudes, aunque eso suponga aceptar más falsos positivos. La estrategia más razonable para el banco es **conservadora**: es preferible molestar puntualmente a un cliente legítimo mediante un bloqueo temporal que permitir que un fraude se consume sin ser detectado.

4.4. Tarea 4 — Curvas ROC y Precision-Recall

4.4.1. Curva ROC

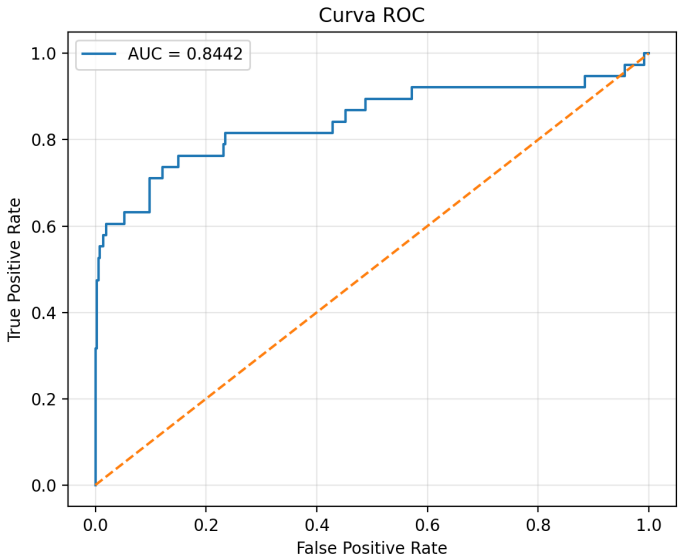


Figura 4.3: Curva ROC del clasificador. AUC-ROC = 0,8442.

4.4.2. Curva Precision-Recall

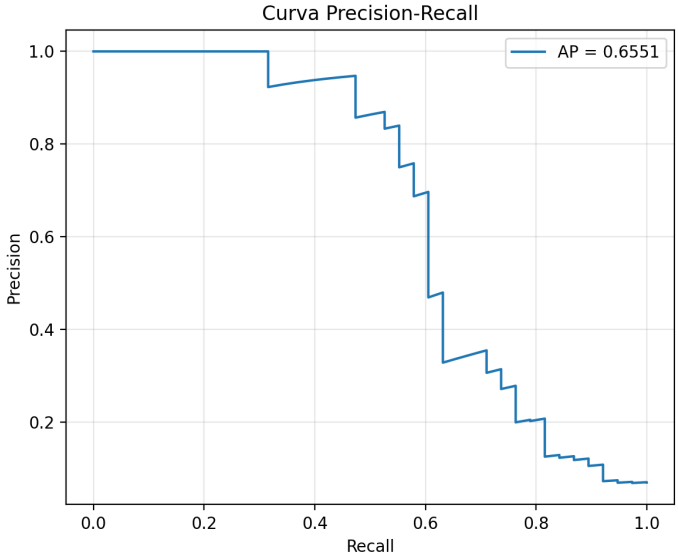


Figura 4.4: Curva Precision-Recall del clasificador. Average Precision = 0,6551.

4.4.3. Discusión

Con un dataset desbalanceado, la curva Precision-Recall resulta más adecuada que la ROC para valorar la capacidad real del clasificador. El motivo es estructural: el *False Positive Rate* de la curva ROC toma como denominador el total de negativos, que en este problema es muy numeroso. Como consecuencia, incrementos apreciables de falsos positivos apenas desplazan el FPR y el AUC-ROC se mantiene elevado (0.8442) pese a que el modelo comete errores relevantes sobre la clase fraude. La curva PR evita este efecto porque la Precisión (TP /(TP+FP)) cae en cuanto aumentan los FP, lo que la hace sensible exactamente a la calidad de recuperación de la clase minoritaria que interesa controlar.

4.5. Decisión final de ingeniería

A la vista de los resultados, se propone fijar $\tau = 0{,}05$ en lugar del umbral por defecto. Con este valor el coste total desciende de 4410 € a 2920 € sobre el conjunto de prueba, una reducción del 34 %. El incremento en falsos positivos (de 1 a 112) es considerable, pero su coste unitario (10 €) es veinte veces inferior al de un falso negativo (200 €), de modo que la operación resulta económicamente ventajosa: se

pasa de 22 FN a 9 FN, y el ahorro por fraudes detectados supera con amplitud el coste adicional por bloqueos erróneos.

Cabe señalar que este umbral se ha determinado exclusivamente a partir de la función de coste definida en la práctica. En un entorno real debería validarse sobre datos de producción y recalibrarse periódicamente si la distribución de transacciones varía.

4.6. Conclusión final

La práctica pone de manifiesto que la elección del umbral de decisión no es una cuestión trivial: fijar $\tau = 0{,}5$ sin análisis previo generó un coste de 4410 €, frente a los 2920 € obtenidos con el umbral óptimo. Hemos verificado que el valor que minimiza el coste total se desplaza hacia la izquierda respecto al punto de equilibrio porque la penalización por FN (200 €) es desproporcionadamente mayor que la de FP (10 €): a medida que se baja el umbral, el ahorro por fraudes adicionales detectados supera el coste extra de los bloqueos erróneos, hasta que ambos efectos se igualan en torno a $\tau^* = 0{,}05$.

Asimismo, se comprueba que en problemas con clases fuertemente desbalanceadas el AUC-ROC puede inducir a conclusiones optimistas, mientras que la curva Precision-Recall refleja con mayor fidelidad la capacidad del modelo para recuperar la clase minoritaria sin disparar el número de falsas alarmas. Ambos resultados subrayan que la evaluación de un clasificador en producción debe incorporar necesariamente los costes asociados al dominio y no limitarse a métricas genéricas.

Capítulo 5

Cuaderno de Laboratorio — Práctica 5: Arquitectura de Redes Neuronales (Perceptrón Multicapa)

Objetivo

Estudiar la influencia de la arquitectura de una red neuronal de tipo Perceptrón Multicapa (MLP) sobre la capacidad de un clasificador para aprender fronteras de decisión no lineales. Se analizan el efecto del número de neuronas por capa, la elección de la función de activación y la profundidad de la red, culminando con la búsqueda de una arquitectura que alcance al menos un 95 % de acierto en el dataset de dígitos escritos a mano.

Introducción: el problema de la no linealidad

En la mayoría de los problemas reales las clases no se pueden separar mediante una frontera plana. Un clasificador lineal (suma ponderada de las entradas seguida de un umbral) produce una única hiperplano de separación. Cuando la geometría de los datos es no lineal —como ocurre con el dataset `make_moons`, cuyos puntos forman dos medias lunas entrelazadas— dicha frontera es inherentemente insuficiente y el modelo cometerá errores sistemáticos independientemente de cuánto se ajuste.

La solución es añadir capas ocultas a la red. Cada capa oculta aprende representaciones intermedias de los datos y, al combinarlas con funciones de activación no lineales, la red puede aproximar fronteras de decisión arbitrariamente complejas.

Esta práctica explora experimentalmente cuántas capas y neuronas son necesarias según la complejidad del problema, y por qué la elección de la función de activación afecta tanto a la velocidad de entrenamiento como a la calidad de la solución.

Todo el código utiliza `scikit-learn` (`MLPClassifier` dentro de un `Pipeline` con `StandardScaler`) y semilla aleatoria fija (`random_state = 42`) para garantizar la reproducibilidad.

5.1. Tarea 1 — El Perceptrón Simple: límites de un modelo lineal

5.1.1. Modelo utilizado

Se genera el dataset `make_moons` con 500 muestras y ruido $\sigma = 0{,}25$, se particiona en 70 % entrenamiento / 30 % prueba (estratificado) y se entrena un `MLPClassifier` sin capas ocultas (`hidden_layer_sizes = ()`). Esta configuración equivale a un perceptrón de una sola neurona de salida: su frontera de decisión es siempre un hiperplano (línea recta en 2D).

5.1.2. Frontera de decisión obtenida

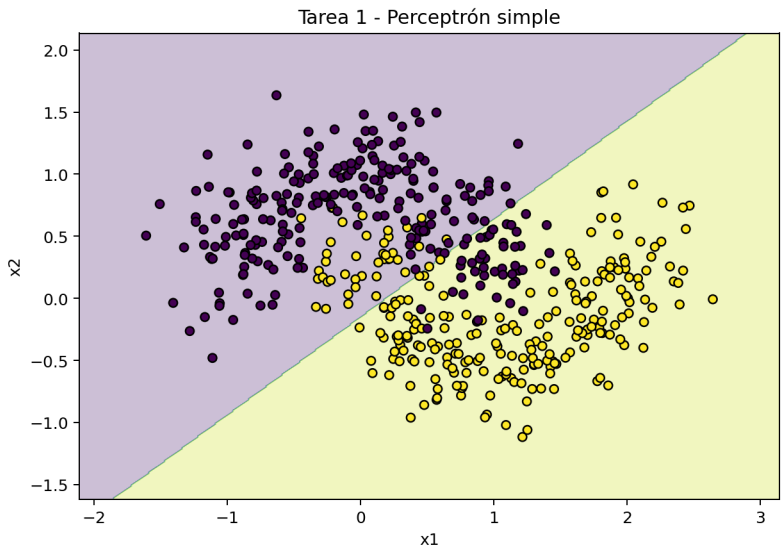


Figura 5.1: Frontera de decisión del perceptrón simple sobre el dataset `make_moons`. La región de color representa la clase predicha; los puntos, las etiquetas reales.

La frontera es, por construcción, una línea recta. El modelo divide el espacio en dos semiplanos, separando correctamente las muestras que caen a un lado, pero es incapaz de capturar la curvatura de las medias lunas.

5.1.3. Matriz de confusión

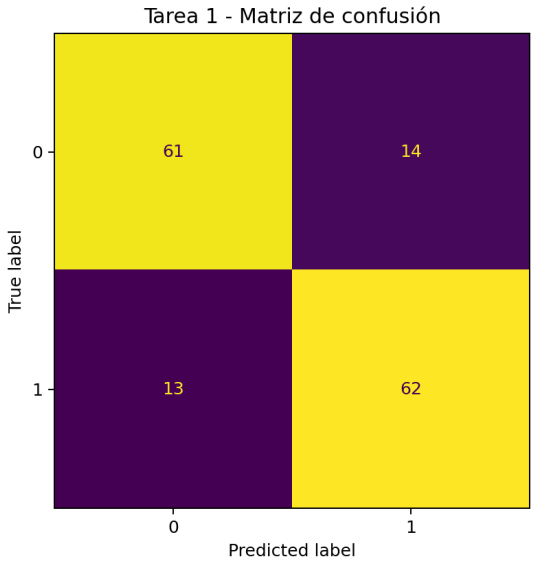


Figura 5.2: Matriz de confusión del perceptrón simple.

5.1.4. Por qué una combinación lineal falla ante esta geometría

El perceptrón sin capa oculta implementa la función $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$, donde σ es una función de activación escalonada. La decisión se produce en la frontera $\mathbf{w}^\top \mathbf{x} + b = 0$, que es un hiperplano. El dataset `make_moons` **no es linealmente separable**: ninguna línea recta puede dejar los puntos de la clase 0 por un lado y los de la clase 1 por el otro sin cometer errores sistemáticos. Por tanto, independientemente del número de épocas o de la tasa de aprendizaje, el perceptrón simple no puede mejorar su rendimiento más allá de lo que permite la mejor línea de separación, que en este caso alcanza una exactitud en torno al 86 %.

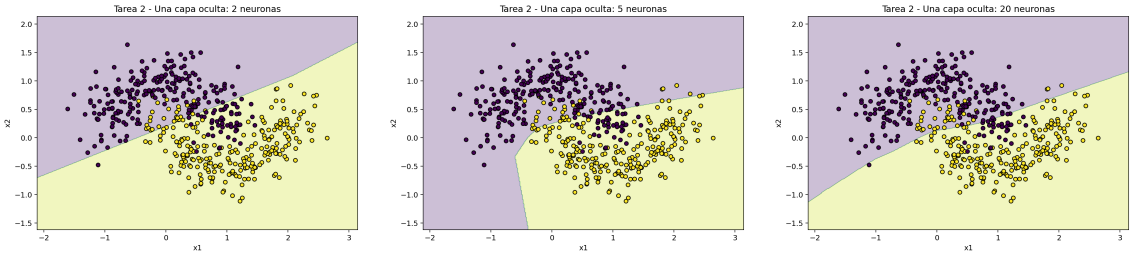
Para superar esta limitación es imprescindible introducir al menos una capa oculta con activación no lineal: la composición de funciones no lineales puede aproximar regionescurvilíneas y, en última instancia, fronteras de decisión de forma arbitraria.

5.2. Tarea 2 — Diseñando la capa oculta: efecto del número de neuronas

5.2.1. Configuraciones evaluadas

Se mantiene la misma partición de datos y se entrena un MLP con una única capa oculta variando el número de neuronas en $\{2, 5, 20\}$. La función de activación es ReLU y el optimizador Adam con hasta 3 000 iteraciones.

5.2.2. Fronteras de decisión



(a) 2 neuronas (b) 5 neuronas (c) 20 neuronas

Figura 5.3: Fronteras de decisión para una capa oculta con distinto número de neuronas.

5.2.3. Tabla de resultados

Tabla 5.1: Comparación de exactitud según el número de neuronas en la capa oculta.

Arquitectura	Acc. entren.	Acc. test	Iteraciones
MLP(2)	0.8429	0.8200	648
MLP(5)	0.8686	0.8400	726
MLP(20)	0.8600	0.8533	406

5.2.4. Análisis: *underfitting* y capacidad del modelo

- **2 neuronas — underfitting claro.** Con solo 2 neuronas ocultas la red puede generar hasta 2 fronteras lineales y combinarlas. La región de decisión es tan poco flexible que la frontera aprendida se parece bastante a la del perceptrón simple: la red es demasiado pequeña para capturar la curvatura de las lunas. Esto se confirma en la Tabla **??**: es la arquitectura con menor exactitud tanto en entrenamiento como en test.

- **5 neuronas — capacidad intermedia.** La frontera empieza a curvarse y a adaptarse a la geometría no lineal, pero aún comete errores en la zona de solapamiento entre clases donde el ruido genera muestras ambiguas. El número de iteraciones hasta la convergencia es el mayor de los tres (726), lo que indica que el optimizador tiene que explorar más para encontrar un mínimo aceptable con esa capacidad limitada.

- **20 neuronas — mejor generalización.** La frontera es claramente no lineal y se ajusta con fidelidad a la forma de las lunas. Se consigue el mayor acierto en test (85.33 %) y la convergencia es la más rápida (406 iteraciones), lo que sugiere que el espacio de parámetros ofrece mínimos locales bien condicionados para este problema. No se aprecia sobreajuste significativo porque la diferencia entre entrenamiento y test es pequeña.

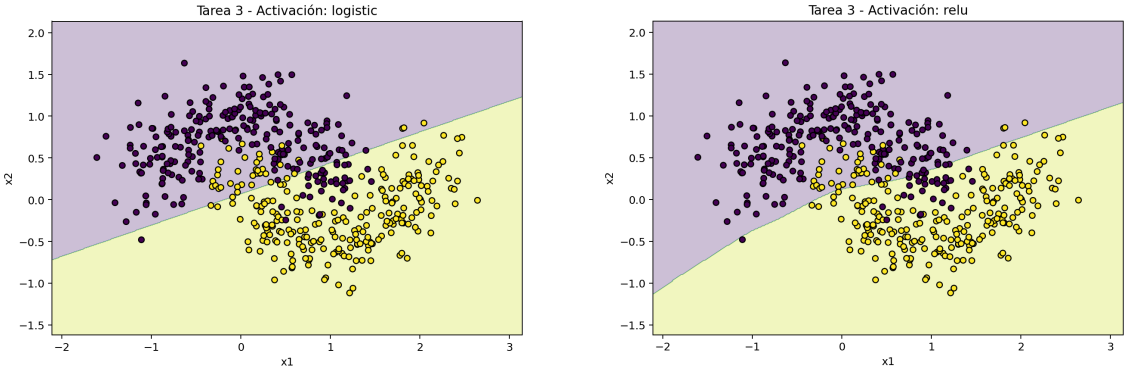
Para un problema de esta complejidad, una capa oculta de 20 neuronas resulta suficiente y equilibrada: añadir más neuronas podría comenzar a memorizar el ruido del conjunto de entrenamiento sin mejorar la generalización.

5.3. Tarea 3 — Funciones de activación: Sigmoide *vs.* ReLU

5.3.1. Configuración del experimento

Se fija la arquitectura en una capa oculta de 20 neuronas y se entrena el mismo modelo dos veces, cambiando únicamente la función de activación: `logistic` (sigmoide) y `relu`. Mantener la arquitectura constante garantiza que cualquier diferencia observada es atribuible exclusivamente a la función de activación.

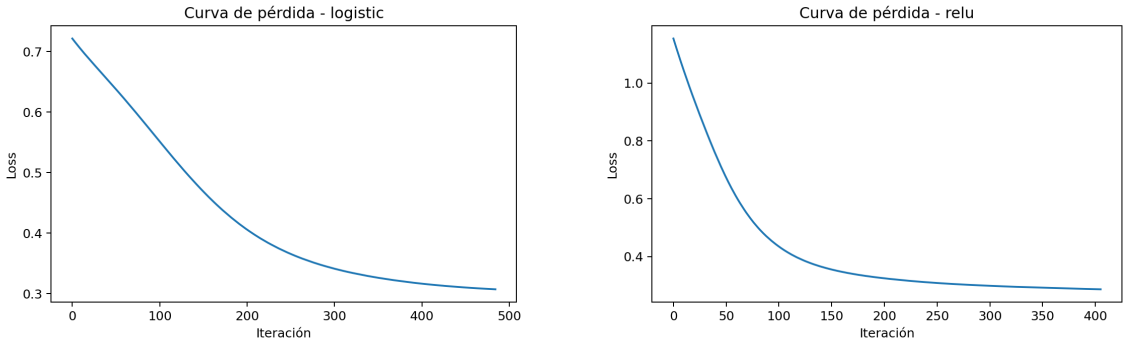
5.3.2. Fronteras de decisión



(a) Sigmoide (`logistic`) (b) ReLU

Figura 5.4: Fronteras de decisión con la misma arquitectura (20 neuronas) y distintas activaciones.

5.3.3. Curvas de convergencia (pérdida de entrenamiento)



(a) Sigmoide (b) ReLU

Figura 5.5: Evolución de la pérdida de entrenamiento (*log-loss*) por iteración.

5.3.4. Tabla comparativa

Tabla 5.2: Comparación entre activaciones Sigmoide y ReLU (arquitectura MLP(20)).

Activación	**Acc. test**	**Iteraciones**	**Loss final**
Sigmoide (`logistic`)	0.8400	485	0.3071
ReLU	0.8533	406	0.2870

5.3.5. Discusión: saturación, gradiente y velocidad de convergencia

Los resultados de la Tabla **??** muestran dos diferencias claras a favor de ReLU:

1. **Menor pérdida final (0.2870 frente a 0.3071)** y **mayor exactitud en test (+1.33 pp).** Aunque la diferencia en accuracy es modesta para este problema, la brecha en la pérdida refleja que ReLU logra una separación más nítida de las clases.

2. **Convergencia más rápida: 406 iteraciones frente a 485 (+16 %).** Esto se explica por el *problema de saturación* de la sigmoide: cuando la entrada a una neurona sigmoide es muy grande o muy pequeña en valor absoluto, la derivada $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ se aproxima a cero, bloqueando el flujo del gradiente hacia las capas anteriores (*vanishing gradient*). ReLU, cuya derivada es exactamente 1 para $z > 0$, no sufre este problema y permite que el gradiente se propague hacia atrás de manera eficiente.

Respecto al aspecto de las fronteras (Figura **??**), la sigmoide produce curvas más suaves y redondeadas, ya que su salida es continua y acotada en $(0, 1)$. ReLU genera fronteras con tramos rectilíneos que se combinan para aproximar la curvatura deseada; el resultado puede ser ligeramente más *angular*, pero en este problema concreto es suficiente y más eficiente desde el punto de vista del entrenamiento.

En conclusión, para redes de profundidad moderada, ReLU es la opción recomendada por su facilidad de optimización, su velocidad de convergencia y su menor pérdida de información por saturación.

5.4. Tarea 4 — El reto de los dígitos escritos a mano

5.4.1. Descripción del dataset

El dataset `load_digits` de scikit-learn contiene 1 797 imágenes de dígitos del 0 al 9 en resolución 8×8 píxeles (64 características). Se reserva el 25 % como conjunto de prueba (partición estratificada, `random_state = 42`), lo que deja 1 347 muestras para entrenamiento y 450 para evaluación. El objetivo es encontrar una arquitectura MLP que supere el 95 % de exactitud en test.

5.4.2. Experimentos realizados

Tabla 5.3: Comparación de arquitecturas MLP en el dataset de dígitos.

Arquitectura	**Acc. entreno.**	**Acc. test**	**Iteraciones**	$\geq$ **95 %**
MLP(20)	1.0000	0.9644	276	✓
MLP(50)	1.0000	0.9711	180	✓
MLP(100)	1.0000	0.9800	147	✓
MLP(30,15)	1.0000	0.9600	161	✓
MLP(50,20)	1.0000	0.9756	131	✓
MLP(64,32)	1.0000	0.9667	110	✓

5.4.3. Matríz de confusión de la arquitectura elegida

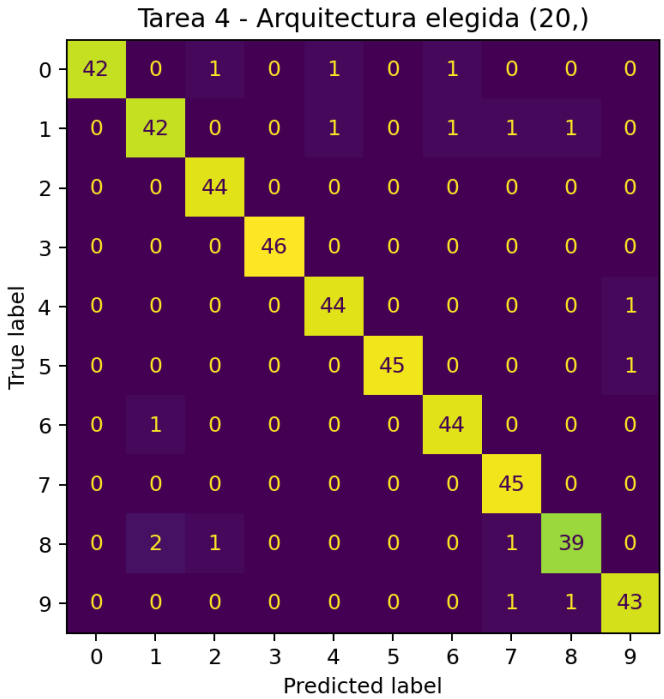


Figura 5.6: Matriz de confusión para la arquitectura elegida MLP(20) sobre el conjunto de prueba. Acc. test = 96.44 %.

5.4.4. Justificación de la arquitectura elegida: MLP(20)

Todas las arquitecturas evaluadas superan el objetivo del 95 %. El criterio de elección no debe ser únicamente maximizar la exactitud, sino encontrar el modelo más sencillo que cumpla el requisito, en línea con el principio de *parsimonia* o la navaja de Occam aplicada al aprendizaje automático.

Se elige **MLP(20): una sola capa oculta de 20 neuronas**, por los siguientes motivos:

1. **Cumple el objetivo**: 96.44 % > 95 %, con un margen razonable.
2. **Menor número de parámetros**: $64 \times 20 + 20 \times 10 = 1\,480$ pesos, frente a los 7 310 de MLP(100). Menos parámetros implica menor riesgo de sobreajuste y modelos más interpretables.
3. **Única capa**: simplifica el paisaje de pérdida y hace la optimización más predecible.

5.4.4.0.1. Red ancha frente a red profunda. La Tabla **??** permite comparar directamente:

- **Una capa grande** (MLP(100)): alcanza el 98 % en test, pero necesita 6 560 parámetros (capa oculta + salida) y converge en 147 iteraciones. Para 64 características de entrada, una sola capa es capaz de aprender las combinaciones relevantes sin necesidad de jerarquizar representaciones.
- **Dos capas más pequeñas** (MLP(30,15), MLP(50,20), MLP(64,32)): las redes de dos capas convergen más rápido en iteraciones (131–161 vs. 276 para MLP(20)), pero no superan necesariamente en exactitud a la red de una capa más pequeña. Para un dataset de dimensionalidad moderada como dígitos 8×8, la profundidad extra no aporta una ganancia proporcional en generalización.

En este problema, la capacidad de representación de una sola capa oculta es suficiente; la profundidad extra no aporta ganancia de generalización apreciable y complica innecesariamente el modelo.

5.5. Conclusión final

Esta práctica ha permitido responder de forma empírica y razonada a las preguntas clave del diseño de redes neuronales:

1. **¿Cuándo usar 0 capas ocultas?** Solo cuando el problema es linealmente separable o cuando se requiere la interpretabilidad máxima y se acepta el coste de no modelar la no linealidad. En cualquier otro caso, un perceptrón simple producirá underfitting estructural que ninguna cantidad de entrenamiento puede resolver.
2. **¿Qué arquitectura usar para problemas sencillos?** Para problemas no lineales de baja a media complejidad (como `make_moons`), una única capa oculta de 10–20 neuronas es suficiente. Aumentar el número de neuronas más allá del necesario no mejora la generalización y puede dificultar la interpretación del modelo.
3. **¿Qué función de activación elegir?** ReLU es la opción por defecto en redes de profundidad moderada: evita la saturación del gradiente, converge más rápido y logra pérdidas finales más bajas que la sigmoide. La sigmoide puede ser útil cuando se necesitan salidas suaves o en problemas con gradientes bien condicionados, pero en general es menos robusta.

4. **Criterio de simplicidad y generalización.** La arquitectura preferida es la **mínima** que supera el umbral de rendimiento exigido, medida en número de parámetros totales. Una red sobredimensionada puede memorizar el conjunto de entrenamiento (acc_train = 1.0 en todos los experimentos de dígitos) sin mejorar el desempeño real. El indicador clave es la brecha entre exactitud en entrenamiento y en test; si es pequeña, el modelo generaliza bien con la arquitectura elegida.

En resumen, el aprendizaje profundo no consiste en apilar capas indefinidamente, sino en encontrar el balance adecuado entre capacidad de representación, velocidad de entrenamiento y generalización. Los experimentos de esta práctica confirman que, para problemas de complejidad media, arquitecturas simples entrenadas con ReLU y escalado previo de las características ofrecen resultados sólidos y eficientes.